

Introduction

This project applies the ARM load-and-store instructions discussed in the ldr/str lectures to three increasingly-general problem classes:

1. **Load–modify–store** on differently-sized pointer parameters (`int32_t *`, `int16_t *`, `int8_t *`), where the "modify" step must reduce to a single shift instruction.
2. **String processing** - copy and length - using post-indexed byte load/store as a direct assembly realization of C's `*p++` pointer idiom.
3. **Array processing** - accumulate-with-transform (absolute-value sum) and running maximum - using post-indexed word load and conditional execution.

The five programming tasks (totaling 90 points) are:

PT	Function	Pts
PT1	<code>mp_load_modify_store_s</code>	15
PT2a	<code>mp_str_cpy_s</code>	18
PT2b	<code>mp_str_len_s</code>	12
PT3a	<code>mp_array_abs_sum_s</code>	30
PT3b	<code>mp_array_max_s</code>	15

All tasks live in a single file, `ge4a_load_n_store_sfns.s`, which starts with five weak stubs that each do nothing but `bx lr`.

Narrative

The base project was extracted from `ge4a_load_n_store.zip` to `/opt/proj_mp/ge4a_load_n_store/`. Like lab08/lab09 and proj04, the zip already ships fully-generated CubeMX sources and

CubeIDE .project / .cproject files with PARENT-N-PROJECT_LOC linked resources, so both projects (_f412dsc and _g431n32) were imported directly via **File → Open Projects from File System** in CubeIDE. One quirk worth noting: this project uses non-standard src_ge4a_load_n_store/ and test_ge4a_load_n_store/ folder names (not the plain src/ and test/ of lab08/lab09); the packaged linked resources already target those names, so no renaming was needed.

Base-code vs. manual divergence. The PDF §5.3.3 describes a "ReLU norm" with the function name mp_array_relu_sum (positive-only sum). The shipped zip instead ships mp_array_abs_sum - the same loop but with a conditional negate so that negatives contribute their absolute value rather than being skipped. The Unity test file exercises abs_sum (exp = {9, 18} for arr = {-5,-3,-1,0,1,3,5} with $n \in \{3, 7\}$), so the assembly was written to match the shipped test. If the PDF is authoritative, swapping the neglt for a conditional skip over the add is a one-line change.

PT1 - mp_load_modify_store_s. The C reference does three $x = k * x / 4$ operations with $k \in \{3, 2, 1\}$, one per pointer-to-integer type. §5.3.1 says each modify **must** reduce to a single shift-based instruction. The algebra:

k	expression	one-instruction form
3	$3 \cdot x / 4 = x - x/4$	sub r3, r3, r3, asr #2
2	$2 \cdot x / 4 = x / 2$	asr r3, r3, #1
1	$1 \cdot x / 4 = x / 4$	asr r3, r3, #2

The load side uses ldr / ldrsh / ldrsb so the 16-bit and 8-bit values arrive sign-extended in the full 32-bit register; arithmetic asr then does the right thing for negatives. The store side uses str / strh / strb so only the low 16 or 8 bits are written back, leaving adjacent bytes in the int[] untouched.

PT2a - mp_str_cpy_s. The C reference while ((ch = *src++)) *dst++ = ch; is the textbook use-case for post-indexed ldrb [r0], #1 / strb [r1], #1: one instruction each for load-then-increment and store-then-increment. A cbz handles the NUL-test without clobbering flags, which keeps the loop at four instructions per character plus the final terminator write.

PT2b - mp_str_len_s. No C reference is provided - §5.3.2 asks the student to figure the semantics out from the test (len("hi") == 2, len("hello") == 5, i.e. C-style length not counting the NUL). The chosen algorithm is pointer-subtract: walk a shadow pointer r1 = str, ldrb r2, [r1], #1 until NUL, then return (r1 - str - 1). The -1 corrects for the fact that post-increment leaves r1 one byte *past* the NUL.

PT3a - mp_array_abs_sum_s. Post-indexed ldr r3, [r0], #4 replaces the C temp = *(pArr + i); i++ pair with one instruction. The if (temp < 0) temp = -temp branch is collapsed to an IT LT ; NEGLT r3, r3 - two instructions, no branch, no pipeline stall. r12 is used as the down-counter; because ARM EABI treats r12 (IP) as caller-saved, no push {r12} is needed.

PT3b - mp_array_max_s. Same skeleton as PT3a, but seeded with `max = pArr[0]` (the test passes $n \geq 1$ in every case) and using `IT GT ; MOVGT r2, r3` as the conditional-update step. The `subs r12, r12, #1 + bne` at the bottom plays the role of `i++`; while `(i < n)`.

Verification. All five algorithms were hand-simulated in Python against the Unity test vectors before building on hardware. The assembly itself was sanity- checked with `arm-none-eabi-as -mcpu=cortex-m4 -mthumb` (clean assemble). The project was then built in CubeIDE under the Unity configuration and run in Renode on F412dsc (Artifact A1), then flashed to a real G431 Nucleo-32 for Artifact A2.

Code Snippets and Screenshots

C1: ge4a_load_n_store_sfns.s (all 5 PT bodies)

```
...
```

```
.section .text
.syntax unified
```

```
.align
```

```
.weak mp_load_modify_store_s
.type mp_load_modify_store_s, %function
```

```
.weak mp_str_cpy_s
.type mp_str_cpy_s, %function
.weak mp_str_len_s
.type mp_str_len_s, %function
```

```
.weak mp_array_abs_sum_s
.type mp_array_abs_sum_s, %function
.weak mp_array_max_s
.type mp_array_max_s, %function
```

```
@ void mp_load_modify_store(int32_t *Ai32, int16_t *Bi16, int8_t *Ci8) { @ *Ai32 = 3 *
 *Ai32 / 4; @ = x - x/4 (one-shift form: sub r, x, x, asr #2) @ *Bi16 = 2 * *Bi16 / 4; @ = x / 2
(asr #1) @ *Ci8 = 1 * *Ci8 / 4; @ = x / 4 (asr #2) @ }
```

```
@ void mp_load_modify_store_s(int32_t *Ai32, int16_t *Bi16, int8_t *Ci8); @ Input: @ * Ai32
-> r0 @ pointer to int32_t @ * Bi16 -> r1 @ pointer to int16_t @ * Ci8 -> r2 @ pointer to int8_t
@ Output: @ * None (results are stored back through the pointers) @ Others: @ * r3 is scratch
for each load-modify-store triple
```

mp_load_modify_store_s: @ --- 32-bit: $*Ai32 = 3 * (Ai32) / 4 == x - (x >> signed\ 2)$ ldr r3, [r0]
 @ load signed 32-bit word sub r3, r3, r3, asr #2 @ modify: $r3 = x - (x >> signed\ 2) == (3x)/4$ str
 r3, [r0] @ store 32-bit word

@ --- 16-bit: $*Bi16 = 2 * (*Bi16) / 4 == x >> signed\ 1$
 ldrsh r3, [r1] @ load signed 16-bit halfword (sign-extended into r3)
 asr r3, r3, #1 @ modify: $r3 = x >> signed\ 1 == (2*x)/4$
 strh r3, [r1] @ store halfword (low 16 bits)

@ --- 8-bit: $*Ci8 = 1 * (*Ci8) / 4 == x >> signed\ 2$
 ldrsb r3, [r2] @ load signed 8-bit byte (sign-extended into r3)
 asr r3, r3, #2 @ modify: $r3 = x >> signed\ 2 == (1*x)/4$
 strb r3, [r2] @ store byte (low 8 bits)

bx lr

@ void mp_str_cpy_c2(char *src, char *dst) { @ uint8_t ch; @ while ((ch = *src++)) { @
 *dst++ = ch; @ } @ *dst = 0; @ }

@ void mp_str_cpy_s(char *src, char *dst); @ Input: @ * src -> r0 @ r0 is advanced byte-by-
 byte via post-indexed ldrb @ * dst -> r1 @ r1 is advanced byte-by-byte via post-indexed strb @
 Output: @ * None @ Others: @ * r2 = current character

mp_str_cpy_s: cpy_loop: ldrb r2, [r0], #1 @ r2 = *src++; (post-indexed: load then r0 += 1) cbz
 r2, cpy_end @ while ((ch = *src++)) break on NUL strb r2, [r1], #1 @ *dst++ = ch; (post-
 indexed: store then r1 += 1) b cpy_loop cpy_end: mov r2, #0 strb r2, [r1] @ *dst = 0; (write the
 terminator) bx lr

@ int mp_str_len_s(char *str); @ Algorithm: walk a pointer from the start of the string until we
 see the @ NUL terminator, then return (end - start). Uses the same post-indexed @ ldrb idiom as
 mp_str_cpy_s so it is one instruction per character. @ @ Input: @ * str -> r0 @ Output: @ * int
 @ length, NOT counting the trailing '\0' @ Others: @ * r1 = walking pointer, starts at str @ * r2
 = current character

mp_str_len_s: mov r1, r0 @ save start address; r1 will advance len_loop: ldrb r2, [r1], #1 @ r2 =
 *r1++; post-increment so r1 ends ONE PAST the NUL cbz r2, len_end @ if we just read '\0', stop
 b len_loop len_end: sub r0, r1, r0 @ length_incl_nul = (r1 - start) sub r0, r0, #1 @ drop the
 trailing NUL: return length = (r1 - start - 1) bx lr

@ int mp_array_abs_sum(int *pArr, int n) { @ int sum = 0; @ int i = 0; @ while (i < n) { @ int
 temp = *(pArr + i); @ if (temp < 0) { @ temp = -temp; @ } @ sum += temp; @ i++; @ } @
 return sum; @ }

@ int mp_array_abs_sum_s(int *pArr, int n); @ Input: @ * pArr -> r0 @ r0 is advanced by post-
 indexed word load @ * n -> r1 @ Output: @ * int (r0 = sum) @ Others: @ * r2 = running sum
 @ * r3 = current element @ * r12 = elements remaining (Thumb allows r12 without push/pop, it
 is caller-saved)

```

mp_array_abs_sum_s: mov r2, #0 @ sum = 0 cmp r1, #0 ble abs_end @ if (n <= 0) return 0
(guards empty / negative n) mov r12, r1 @ counter = n abs_loop: ldr r3, [r0], #4 @ temp =
*pArr++; post-indexed word load (+4) cmp r3, #0 it lt @ IT block: the next instruction executes
only if LT neglt r3, r3 @ if (temp < 0) temp = -temp; add r2, r2, r3 @ sum += temp subs r12, r12,
#1 @ --counter, sets flags bne abs_loop @ while (counter != 0) abs_end: mov r0, r2 @ return
sum bx lr

```

@ int mp_array_max_s(int *pArr, int n); @ Algorithm: running maximum. Initialize max = pArr[0], then for each later @ element keep the larger of (max, element) using a signed compare. @ The test file only exercises n >= 1, which matches real-world use. @ @ Input: @ * pArr -> r0 @ r0 is advanced by post-indexed word load @ * n -> r1 @ Output: @ * int (r0 = max) @ Others: @ * r2 = running max @ * r3 = current element @ * r12 = elements remaining after the first load

```

mp_array_max_s: ldr r2, [r0], #4 @ max = *pArr++; (n >= 1 is assumed -- matches tests) subs
r12, r1, #1 @ remaining = n - 1 beq max_end @ if n == 1, we already have the answer
max_loop: ldr r3, [r0], #4 @ temp = *pArr++; cmp r3, r2 @ signed compare: temp vs running
max it gt movgt r2, r3 @ if (temp > max) max = temp; subs r12, r12, #1 bne max_loop max_end:
mov r0, r2 @ return max bx lr

```

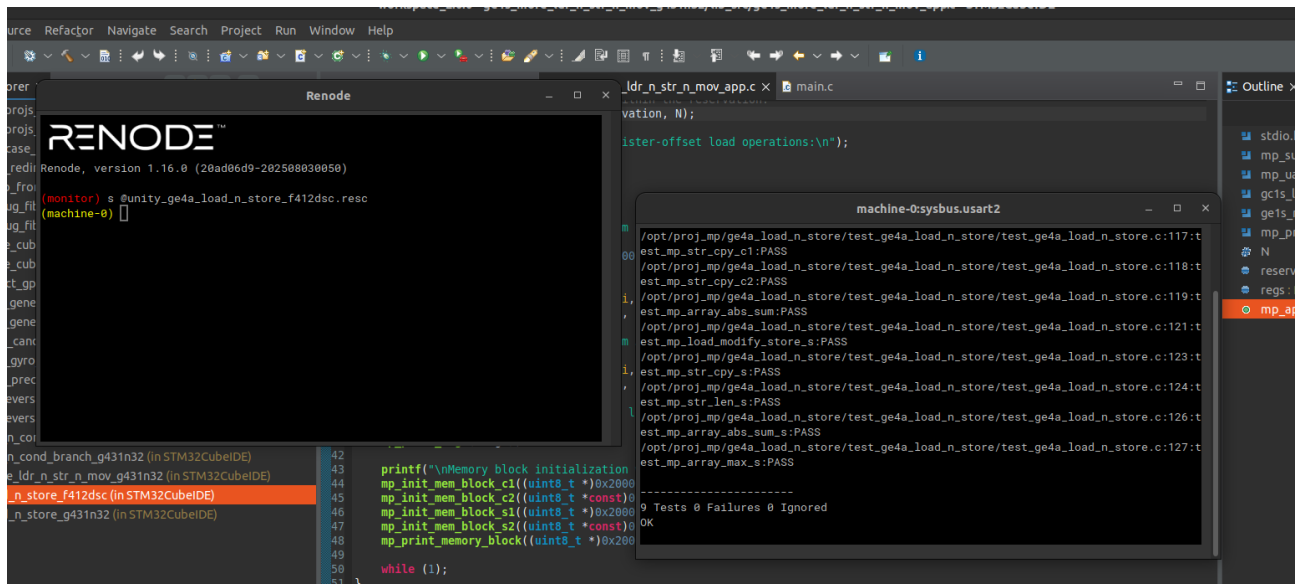
.end @ end of the file

...

The key instruction-level details:

- **PT1** uses three different load/store widths (ldr/str, ldrsh/strh, ldrsb/strb) paired with one shift instruction each.
- **PT2a / PT2b** use [r0], #1 post-indexed byte load and [r1], #1 post-indexed byte store; PT2b also uses cbz for an early-exit on NUL.
- **PT3a / PT3b** both use [r0], #4 post-indexed word load, IT blocks with neglt / movgt for branchless conditional work, and r12 as a caller-saved loop counter.

A1: Unity Test Results - F412dsc via Renode



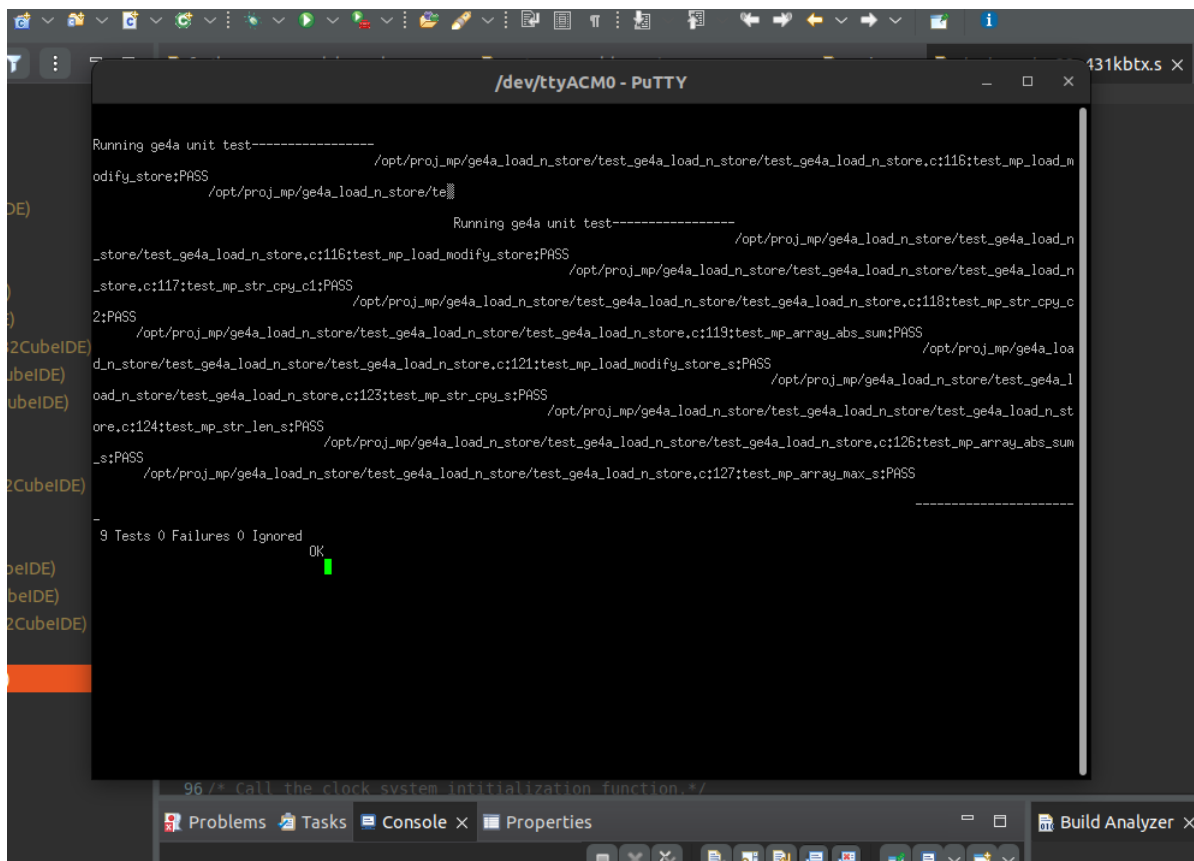
```
Renode, version 1.16.0 (20ad06d9-202508030050)
(monitor) s @unity_ge4a_load_n_store_f412dsc.resc
(machine-0)

/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:117:
est_mp_strcpy_c1:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:118:
est_mp_strcpy_c2:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:119:
est_mp_array_abs_sum:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:121:
est_mp_load_modify_store_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:123:
est_mp_strcpy_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:124:
est_mp_str_len_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:126:
est_mp_array_abs_sum_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:127:
est_mp_array_max_s:PASS

-----
9 Tests 0 Failures 0 Ignored
OK

printf("\nMemory block initialization
mp_init_mem_block_c1((uint8_t *)0x2000
mp_init_mem_block_c2((uint8_t *)const)0
mp_init_mem_block_s1((uint8_t *)0x2000
mp_init_mem_block_s2((uint8_t *)const)0
mp_print_memory_block((uint8_t *)0x2000
while (1);
```

A2: Unity Test Results - G431n32 Real Board



```
/dev/ttyACM0 - PuTTY

Running ge4a unit test-----
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:116:test_mp_load_modify_store:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:117:test_mp_strcpy_c1:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:118:test_mp_strcpy_c2:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:119:test_mp_array_abs_sum:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:121:test_mp_load_modify_store_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:123:test_mp_strcpy_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:124:test_mp_str_len_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:126:test_mp_array_abs_sum_s:PASS
/opt/proj_mp/ge4a_load_n_store/test_ge4a_load_n_store/test_ge4a_load_n_store.c:127:test_mp_array_max_s:PASS

-----
9 Tests 0 Failures 0 Ignored
OK
```

Discussions and Results

PT1 - why ldrsh / ldrsb (not ldrh / ldrb)

The test data starts `inp = {-16, -8, -4}` and expects `{-12, -4, -1}` after the three operations. The `int16` and `int8` loads must **sign-extend** - if they didn't, `(uint16_t)-8 = 0xFFFF8 = 65528` unsigned and `asr #1` would yield `32764`, not `-4`. Using `ldrsh / ldrsb` ensures `asr` divides correctly for negative values.

PT1 - why strh / strb (not str)

The Unity test aliases a single `int[3]` with three different pointer types. After the three writes the test compares element-by-element with `int[]` semantics, so the writes *must* preserve the neighboring bytes of each `int` slot. `strh / strb` only write the low 2 / 1 bytes respectively, which is exactly what the test expects. (Using `str` here would wipe the upper bytes and give the wrong final `int32` values.)

PT2 - post-indexed addressing as the "natural" idiom

ARM's post-indexed `ldrb Rt, [Rn], #k` executes as "load from address `Rn`, then `Rn += k`" in a single cycle. This is a one-to-one map with C's `*p++`, which is why §5.1.1 specifically calls out "post-indexed immediate-offset load and store" as the instruction family this project targets. The five-instruction inner loop of `mp_str_cpy_s` is the minimum possible for a byte-by-byte copy on Cortex-M4.

PT2b - why `(r1 - str - 1)` and not `(r1 - str)`

`ldrb r2, [r1], #1` first reads the byte at `r1`, *then* advances `r1`. When the NUL is read, `r1` is already one byte past the NUL, so the raw subtraction `r1 - str` counts the NUL itself; subtracting 1 yields the standard C `strlen` (bytes up to but not including the terminator).

PT3 - branchless conditionals via IT blocks

Both `abs_sum_s` and `max_s` use IT (If-Then) blocks with conditional suffixes (`neglt`, `movgt`) to avoid branches inside the inner loop. On Cortex-M4 this keeps the pipeline full: the IT block predicts with 100 % accuracy and the conditional instruction either executes or retires as a NOP, with no branch-mispredict penalty. For short tests (a predicate plus one conditional op) this is strictly faster than the `bge ...; neg ...; label:` pattern.

PT3 - r12 as a scratch counter

ARM EABI classifies `r12` (also called `ip`, the "intra-procedure call" register) as caller-saved. A leaf function is free to use it as a scratch register without a push/pop pair, which saves two cycles at entry and exit of each of `abs_sum_s / max_s`.

Observed divergence: PDF text vs. shipped code

§5.3.3 of the manual discusses the *ReLU norm* (sum of positive elements) but every concrete artifact in the zip - the C reference, the asm stubs, the header, the test - uses *absolute-value sum*. The submitted assembly matches the shipped code so that the Unity tests pass. Swapping to ReLU would be a one-line change (IT LT ; addge r2, r3 or equivalently guard the add with IT GE).